CHAPTER

ONE

ML-AUGMENTED ALGORITHM ANALYSIS VIA COMBINATORIAL STRUCTURE GENERATION

1.1 Introduction

Given the inherent hardness of many combinatorial optimization problems (COPs), a primary objective in the field is to develop approximation algorithms that combine tractable runtimes with the tightest possible proven optimality bounds.

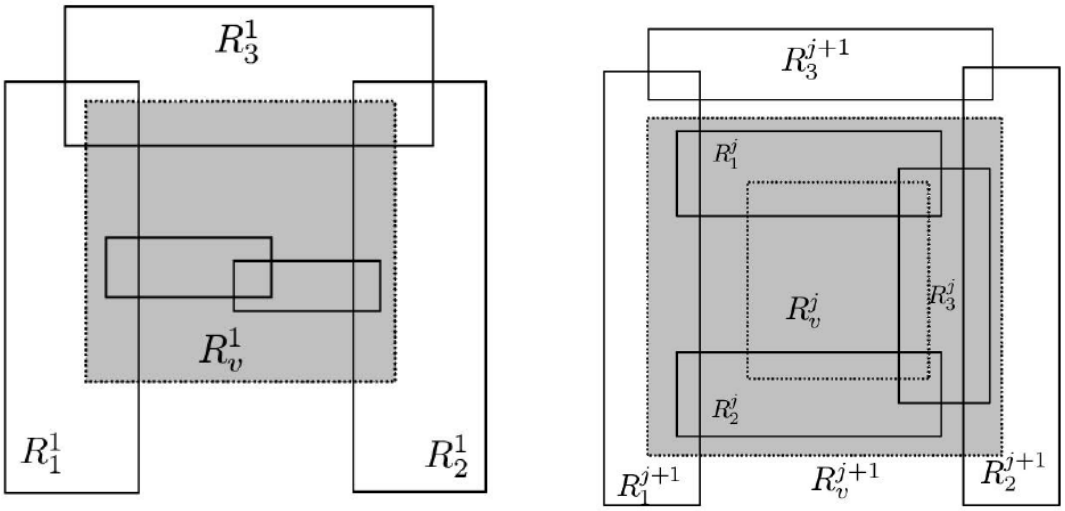
For COPs that can be formulated as integer linear programs (ILPs), a common technique to design an approximation algorithm is to first relax the integrality constraints of the ILP. By solving the resulting linear programming relaxation (LPR) and developing a rounding scheme, one can quickly find a feasible solution for the original problem. The remaining challenge is to bound the optimality of solutions produced by this scheme.

We will consider the tightness of such bounds for a geometric COP: Maximum Independent Set of Rectangles (MISR). Specifically, we investigate the discrepancy between theoretically proven worst-case optimality bounds and known worst-case instances. When these values are not tight, it remains unclear whether the analytical proof can be improved, or if there exists an undiscovered instance that yields poorer solution quality.

When an approximation algorithm is based on a rounded LPR solution, a worst-case

instance can be pinpointed by examining the gap between the ILP and LPR solutions—the integrality gap. Identifying such instances, whether based on an LPR or otherwise, is an exceedingly difficult task that often requires significant ingenuity to construct intricate, repeated structures.

For example, Chuzoy et al designed a repeated gadget structure, depicted in Figure 1.1, to yield a worst-case instance for the optimality bounds of an LPR-based rounding scheme on MISR.



(a) Initial instance (b) $i+1$ gadget

Figure 1.1: Chuzoy initial instance and $i+1$ gadget

The construction takes the first instance in Figure 1.1a, rotates it 90 degrees, and adds the three-rectangle gadget in Figure 1.1b. When repeated, the resulting configuration yields a ratio of LPR to ILP solution arbitrarily close to $3/2$. Although aesthetic, constructions such as this are difficult to conceive, requiring experts to expend intense focus on a specific problem over long time horizons.

In this chapter, we investigate how we can use reinforcement learning techniques to automate the identification of worst case instances such as this. More broadly, we aim to design techniques to discover combinatorial structures of interest.

Applications in this domain hold enormous potential, particularly with a view towards algorithmic analysis. If ML techniques can augment the ability of algorithm engineers in this regard, which are intensive in their demand of time and expertise, it represents a promising avenue to greatly accelerate algorithmic research. Similar research in this vein is a focus in many leading AI research centres. A close parallel

is seen in Google's focus on geometric mathematical problem solving with their AlphaGeometry [17] agent, which has recently neared gold medal level performance on geometric international math Olympiad problems.

This work required a long exploration phase to develop our methodology, as reinforcement learning techniques are still in their infancy in terms of applications in this domain. We will begin with some simpler problems than the integrality gap ones mentioned above, highlighting how we focused on these to overcome a number of relevant research challenges as we work towards the larger goal of worst-case instance discovery. First, we will define some of these problems more formally, followed by some relevant work from the literature.

1.2 Preliminaries

Below are outlined some of the relevant combinatorial optimisation problems in this chapter.

1.2.1 The N-Queens Problem

The N-Queens problem is a classic combinatorial constraint satisfaction problem that has long served as a fundamental benchmark in algorithmic design. The objective is to place exactly N chess queens on an $N \times N$ chessboard such that no two queens threaten each other. In accordance with the rules of chess, this requires that no two queens share the same row, column, or diagonal. While originally posed as a mathematical puzzle, it exemplifies spatial arrangement under strict non-overlapping constraints, sharing conceptual ties with independent set problems on highly structured graphs.

To formulate the N-Queens problem as an Integer Linear Program (ILP), we define a binary decision variable $x_{i,j} \in \{0, 1\}$ for each cell on the board, where $i, j \in \{1, \ldots, N\}$. The value $x_{i,j} = 1$ indicates that a queen is placed in the i-th row and j-th column, and 0 otherwise. Because this is primarily a feasibility problem, the objective can be framed as maximizing the number of placed queens: $\max \sum_{i=1}^{N} \sum_{j=1}^{N} x_{i,j}$, with the goal of reaching exactly N. The placement rules are enforced through a series of constraints. Every row and every column must contain exactly one queen, yielding the constraints $\sum_{j=1}^{N} x_{i,j} = 1$ for all rows i, and $\sum_{i=1}^{N} x_{i,j} = 1$ for all columns j. Further-

more, to prevent diagonal attacks, no more than one queen can occupy any positive diagonal ($\sum_{i-j=k} x_{i,j} \leq 1$ for all $k \in \{-(N-1), \ldots, N-1\}$) or negative diagonal ($\sum_{i+j=k} x_{i,j} \leq 1$ for all $k \in \{2, \ldots, 2N\}$).

The Linear Programming (LP) relaxation of the N-Queens formulation is obtained by relaxing the binary restriction, allowing the variables to take fractional values such that $x_{i,j} \in [0, 1]$. While the strict integer version forces a discrete search space, the LP relaxation can be solved efficiently in polynomial time. However, this relaxation often yields highly fractional and symmetric solutions (for instance, placing uniform fractional values across the board) that satisfy all row, column, and diagonal sums without translating to valid integer placements. This highlights a classic integrality gap often encountered when relaxing strict spatial puzzles.

1.2.2 Maximum Independent Set on Interval Graphs

The Maximum Independent Set (MIS) problem on interval graphs is a classic combinatorial optimisation problem. Given a set of intervals on the real line, the goal is to find a maximum-cardinality subset of intervals that are pairwise disjoint (i.e., no two intervals intersect). In the context of the corresponding interval graph, this equates to finding the largest set of vertices such that no two share an edge. Because interval graphs are perfect graphs, the MIS problem can be solved efficiently in polynomial time, often via a simple greedy strategy such as selecting the interval with the earliest end-time.

This problem can be formulated naturally as an Integer Linear Program (ILP). Let $\mathcal{I}$ be the set of given intervals. We introduce a binary decision variable $x_I \in \{0, 1\}$ for each interval $I \in \mathcal{I}$, where $x_I = 1$ if the interval is included in our independent set, and 0 otherwise. The objective is to maximise the size of the independent set: $\max \sum_{I \in \mathcal{I}} x_I$. To ensure that no two selected intervals overlap, we enforce the constraint that for any point p on the real line, at most one chosen interval can cover it. Thus, for every continuous point p (or effectively, every maximal clique in the interval graph), we have the constraint $\sum_{I \ni p} x_I \leq 1$. The Linear Programming (LP) relaxation of this problem is obtained by relaxing the integrality constraint on the variables, allowing them to take fractional values such that $x_I \in [0, 1]$.

1.2.3 Maximum Independent Set of Rectangles

The Maximum Independent Set of Rectangles (MISR) problem is a natural two-dimensional generalisation of the interval MIS problem. Given a set of axis-parallel rectangles in the plane, the objective is to find a maximum-cardinality subset of rectangles that are mutually non-overlapping. While the one-dimensional interval version is easily solvable in polynomial time, moving to two dimensions introduces significant complexity, making the problem strongly NP-hard. It has widespread applications in fields requiring spatial packing, such as VLSI design, map labeling, and resource allocation.

The ILP formulation for MISR closely mirrors the one-dimensional case. Let $\mathcal{R}$ denote the set of axis-parallel rectangles. We assign a binary decision variable $x_R \in \{0, 1\}$ to each rectangle $R \in \mathcal{R}$. We wish to maximise the total number of selected rectangles: $\max \sum_{R \in \mathcal{R}} x_R$. The independence condition is enforced by requiring that no point p in the plane is covered by more than one selected rectangle. This gives the constraint $\sum_{R \ni p} x_R \leq 1$ for all points p. Just as before, the LP relaxation is formulated by dropping the strict binary requirement, allowing the variables to take continuous fractional values $x_R \in [0, 1]$. This relaxation is frequently used as a foundational step in approximation algorithms to bound the optimal solution.

1.2.4 Reinforcement Learning Fundamentals

Reinforcement learning (RL) provides a formal mathematical framework for solving sequential decision-making problems. At the core of this framework is the *agent*, the autonomous decision-making entity that interacts with and learns from an environment. These problems are rigorously formulated as Markov Decision Processes (MDPs). An MDP is defined by a 5-tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where:

- $\mathcal{S}$ is a set of valid states, defining the state space.
- $\mathcal{A}$ is a set of available actions, defining the action space.
- $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the state transition probability function. Here, $\mathcal{P}(s' \mid s, a)$ denotes the probability of transitioning to state s' given that action a is taken in state s. This formulation assumes the Markov property, whereby state transi-

tions depend solely on the current state and action, independent of the sequence of prior states.

- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ is the reward function, which dictates the expected immediate scalar reward R_t received upon transitioning from state s to state s' via action a.
- $\gamma \in [0, 1]$ is the discount factor, which weights the present value of future rewards. This guarantees mathematical convergence of infinite sums in continuing tasks and regulates the agent's preference for immediate versus delayed gratification.

The behaviour of the agent within an MDP is formalised by a policy, denoted as π. A deterministic policy directly maps states to actions ($\pi : \mathcal{S} \to \mathcal{A}$). More generally, a stochastic policy, $\pi(a \mid s)$, defines a probability distribution over actions $a \in \mathcal{A}$ given a state $s \in \mathcal{S}$.

The central objective of the agent is to discover an optimal policy that maximises the expected cumulative discounted reward, termed the *return*. The return from time step t is denoted G_t and is defined as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

In finite, episodic tasks, the sequence naturally concludes at a terminal state at time T. The final scalar feedback received upon transitioning into this terminal state is referred to as the *terminal reward*, which often encodes the overarching success, failure, or final objective value of the trajectory.

To evaluate the efficacy of a specific policy π, two primary value functions are defined. The state-value function, $V^\pi(s)$, expresses the expected return when starting in state s and following policy π thereafter:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

Similarly, the action-value function, $Q^\pi(s, a)$, specifies the expected return starting from state s, executing an initial action a, and subsequently following policy π:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

Solving an MDP equates to identifying an optimal policy, π^*, that achieves the maximum possible value for all states, such that $V^{\pi^*}(s) \geq V^{\pi}(s)$ for all $s \in \mathcal{S}$. To achieve this, modern RL algorithms are broadly categorised into two paradigms. **Value-based** methods seek to indirectly derive an optimal policy by estimating optimal value functions (e.g., iteratively updating numerical estimates of the optimal action-value function, $Q^*(s, a)$). Conversely, **policy-based** methods explicitly parameterise the policy π and optimise it directly via gradient ascent on the expected return, without fundamentally requiring a value function. A prominent subset of algorithms, known as Actor-Critic methods, hybridises these approaches by concurrently learning a parameterised policy (the actor) and a value function (the critic) to stabilise and guide the learning updates.

1.3 Related Work

A key motivator driving the adaptation of our approach is the paper "Constructions in Combinatorics via Neural Networks" [19]. This approach utlised a binary encoding of graph adjacency lists, and leverages the Cross-Entropy Method (CEM) to find multiple counterexamples to open conjectures in extremal graph theory. These results are reproduced in [8], and their hyperparameter sensitivity experiments highlight the robustness of the CEM RL method.

Work by Ventosa et al. [18] finds further counterexamples in spectral graph theory using Wagner's methodology, and Abbas et al. [9] use the AlphaZero [15] RL agent to find optimal constructions for a classical extremal graph theory problem originally studied by Erdős et al. [5].

Wagner continues this line of research in two further works [2], [7], using more computationally demanding RL techniques in the latter, and an innovative small transformer architecture in the former, which we later utilise.

The ultimate objective of this research is the development of ML-enabled approaches to analyse algorithms by identifying worst-case inputs. Traditional approaches often rely on manually constructed extremal cases, but alternative methodologies can be found in the literature. Some examples lie in the work of Smith-Miles et al. [16] and Bossek et al. [1], who propose the systematic mapping of an "instance space." Instead

of focusing on isolated combinatorial structures with specific properties, this approach characterises instances based on their relative difficulty for a given algorithm. By extracting structural features from these challenging instances and projecting them into a two-dimensional space, researchers can visualize an algorithm's "footprint" - the specific region where its performance is statistically likely to degrade. This visual representation allows for a more strategic selection of algorithms and provides deeper insight into the structural patterns that trigger algorithmic failure.

To populate these instance spaces with a sufficiently diverse range of challenging data, recent research has increasingly employed evolutionary algorithms designed for diversity rather than simple convergence. Quality Diversity (QD) algorithms [14] are particularly effective in this regard, as they identify high-quality solutions across various niches of the problem space. When applied to combinatorial optimization problems such as the Traveling Thief Problem [12], QD enables the generation of a broad spectrum of instances that ensure a more rigorous and comprehensive evaluation of an algorithm's limitations. Similarly, Evolutionary Diversity Optimization (EDO) has been utilized to evolve sets of high-quality solutions for the Traveling Salesman Problem (TSP) [4; 3; 11; 10] and to generate synthetic TSP instances with targeted properties [6]. By leveraging these diversity-driven search techniques, it becomes possible to systematically probe the boundaries of algorithmic efficiency and identify the structural nuances that define the worst-case performance landscape.

1.4 Methodological Development

As mentioned in Section 1.1, this section outlines the various challenges encountered during this research. Understanding these difficulties is essential for contextualizing our methodological choices. A primary point of reference for this work is *Constructions in Combinatorics via Neural Networks* [], which utilized a reinforcement learning (RL) methodology to refute several open conjectures in extremal graph combinatorics. Our experiments aim to extend this approach by generating results for geometric COPs. Before delving into implementation details, we briefly review the learning techniques employed.

1.4.1 Appropriate Learning Techniques

Supervised learning techniques are poorly suited for the discovery of worst-case instances in COPs. For example, considering instances of the Maximum Independent Set of Rectangles (MISR) with a large integrality gap, there is a severe scarcity of unique examples exhibiting a non-zero gap. Consequently, constructing an adequate training dataset for supervised learning is infeasible. This limitation naturally points towards RL. While Wagner et al. [] employ the cross-entropy method, we also experiment with several other contemporary RL algorithms, including DQN [], Actor-Critic [], and PPO [], as well as PatternBoost [], a transformer-based method recently introduced by Wagner et al. for similar domains. A common requirement across all these RL methods is their deployment within a carefully designed Markov Decision Process or RL "environment." The framework used to translate our COPs into a format amenable to RL is critical, as the precise formulation of these environments forms the technical core of our approach.

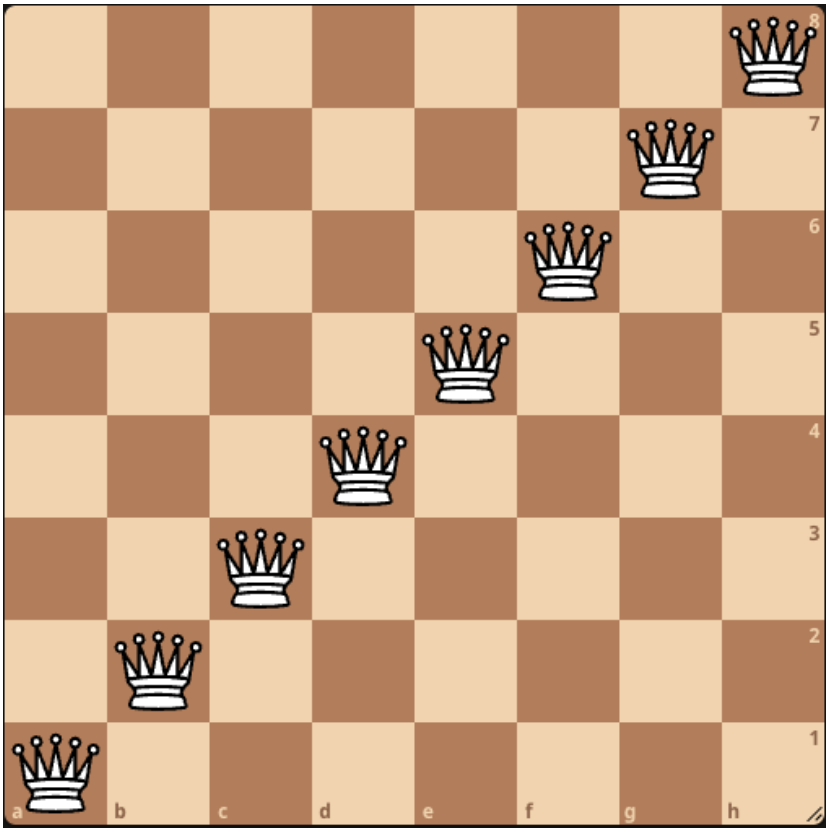
1.4.2 Problem Representation

Designing suitable state and action representations is central to the successful application of RL. State and action formulations are intrinsically linked, and our preliminary experiments on the N-Queens problem proved particularly instructive in navigating these design choices. The central consideration is clear: expansive action and state spaces require prohibitive amounts of data for the RL agent to isolate meaningful training signals. Constraining these spaces significantly improves sample efficiency, a critical requirement given the massive instance spaces inherent to COPs. Furthermore, the representation must facilitate broad exploration of the problem space in a minimal number of steps. Longer episodes exacerbate the credit assignment problem—a fundamental challenge in RL where it becomes difficult to determine which specific actions in a lengthy sequence are responsible for the final reward.

To illustrate this, consider the following RL formulations for the N-Queens problem. If the state is defined as a collection of n queens sequentially placed on the board, this yields a state space of size $O(n^2)$, and an action space of n^2. A full episode is completed in n steps.

We can further refine these spaces by leveraging domain-specific insights. By recog-

Figure 1.2: Example configuration of $N = 8$ queens.



nising that a valid configuration cannot have multiple queens in the same column, we can restrict placements column-wise, shrinking the state space to $O(n^2)$ and action spaces to size n.

Extending this logic to both rows and columns yields our most efficient representation. We initialize the environment with a full board of n queens, strictly assigning one queen per row and one per column. Actions then become permutations of the columns (i.e., swapping the positions of two queens). Consequently, the state representation simplifies to an n-dimensional vector, where the i-th index represents a column and its integer value indicates the row occupied by the queen. For example, the configuration depicted in Figure 1.2 is represented by the following permutation:

$$[1, 2, 3, 4, 5, 6, 7, 8]$$

Under this permutation-based formulation, the state space size is drastically reduced to exactly n, while the action space increases to $\binom{n}{2}$. Although the action space is larger, this formulation permits an agent to move freely through this more densely

represented state space. Empirically, this representation yielded much better performance than the previous versions.

With this permutation-based representation in hand, we continue with another crucial consideration in our experimental design; reward.

1.4.3 Objective Function Design

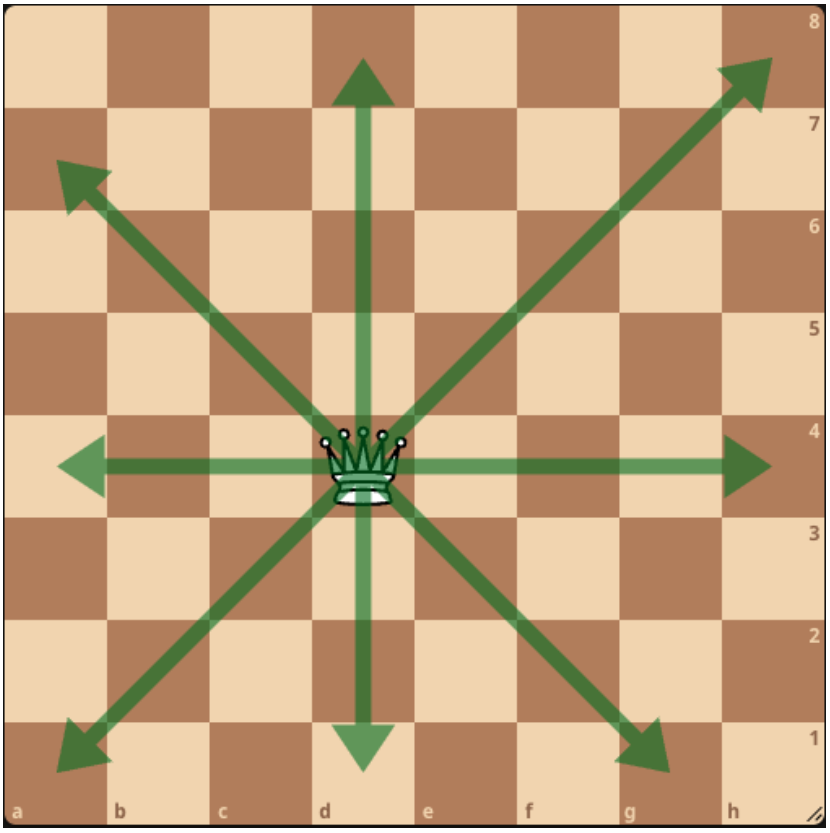
The objective function which an RL agent aims to optimise is also known as the reward function. This function is central to RL, and deviations from straightforward rewards can result in unexpected experimental failures. Using simple goal-focused reward functions represent another interesting challenge in this domain.

Consider once again the problem of finding MISR instances with maximum integrality gap. In this pursuit, we define our reward as the ratio between the ILP and LPR solution. However, we quickly found that RL agents were often receiving no reward at all - the space of instances with a non-zero integrality gap (or ratio equal to 1, in terms of our reward) was extremely sparse. Reward is the basis upon which RL agents learn, and as a result our early RL agents were not discernibly learning anything.

For the N-Queens problem, a framework where reward is given only when an agent reaches a correct configuration encounters similar learning difficulties. This is also referred to in RL nomenclature as "terminal reward"; RL settings where no reward signal is received until the end of an episode. In the case of N-Queens, however, we learned to design a smooth reward function defined by the negative of the number of queen "conflicts". Figure 1.3 depicts the attack pattern of a queen; if a pair of queens are placed in each others attack pattern, this is considered a "conflict".

This reward function provides RL agents with a learning signal at every step, reducing credit assignment issues, and allowing agents to quickly move towards feasible configurations. Tampering with the reward, however, brings with it many other experimental risks. In the case of MISR, early state representations infrequently yielded a LP/ILP ratio of 1.25 (attributable to a simple $5-$cycle) at best. Modifying reward as per other approaches in the literature, by giving extra reward for achieving configurations with a ratio greater than 1.0, or conversely, a penalty for configurations with a ratio equal to 1.0, failed to improve agent performance. We also experimented with adding a "jitter" to rectangle coordinates, which would capture configurations

Figure 1.3: Illustration of queen attack pattern.



geometrically close to a single instance. This too, did not noticeably improve agent performance, while also negatively impacting experimental runtime by enforcing the solving of many more ILP and LPR problems. This severely limits the ability of our approach to scale up to larger problems.

1.4.4 Reinforcement Learning Agents

Along with previous considerations, selection of RL learning methodology (or RL Agent, as referred to in the literature) is yet another crucial aspect in successful implementation. The preliminary approaches which are widely available in RL packages are:

- Deep-Q-Learning (DQN)
- Actor-Critic (A2C)
- Proximal Policy Optimisation

We tested more complex methodologies with preliminary state representations for

MISR, including AlphaZero[], but without an efficient state representation, even this complex method yielded no improvement. The importance of RL agent choice was highlighted starkly in our N-Queens breakthrough experiments, as depicted in Figure 1.4.

Figure 1.4: Experimental Results for varying values of N

In these experiments, only PPO could consistently find feasible configurations as N increased. All three tested agents exhibit good performance in standard RL benchmarks, but this setting seems to exhibit a level of volatility or difficulty not present in such benchmarks.

At this stage, the delicate balance required for these RL experiments was clear; A combination of efficient state representation, non-sparse reward (where possible), and suitable learning technique, were sufficient and necessary in order to achieve good experimental results. We reorganised our focus to adopt the permutation-based representations which had proven successful on more simple combinatorial problems, and combined them with RL agent techniques which have already shown their efficacy in domains involving the generation of specific combinatorial structures. This focus yielded our most promising results. We first outline the final experimental configuration before detailing these results.

1.5 Final Experimental Configuration

Before outlining results on the individual final problems, we outline the common framework between them which was most capable across the set. Reward functions for each problem are

1.5.1 Learning Approaches

Cross-Entropy Method We adopt two approaches as seen in works by Wagner in relation to constructions in extremal combinatorics. The first is the Cross-Entropy (CE) method, used to great effect by Wagner [] to create graphs which act as counterexamples to some long-standing conjectures in graph theory.

By setting the reward to reflect how close a generated structure comes to breaking a theoretical bound, the CE method efficiently guides the network toward discovering novel mathematical constructions.

The algorithm operates in a continuous loop until a set number of iterations is reached:

1. **Sample:** Generate a batch of different neural networks by pulling their parameters from a probability distribution (typically a Gaussian curve).
2. **Evaluate:** Test each network's generated policy or structure and record the reward it earns.
3. **Filter:** Select the top percentage of networks with the highest rewards. These are the "elite" samples.
4. **Update:** Adjust the original probability distribution so it more closely matches the parameters of those elite networks.

By repeating this cycle, the algorithm gradually shifts its search toward parameter regions that yield the highest rewards, effectively training the network to produce optimal policies. This makes it highly useful for problems with complex or difficult-to-calculate reward systems.

(Note: Despite the name, this method is not directly related to the cross-entropy loss function used in standard supervised learning. The name simply refers to how the algorithm measures the mathematical distance between probability distributions during the update step.)

PatternBoost Method We also employ the PatternBoost framework, a hybrid approach introduced by Charton et al. [], designed to generate complex mathematical constructions.

This technique somewhat stretches our classification as an "RL agent", but nonetheless sits within our RL-designed problem representation in the same way as other agents. It also relies on some of the same "elite" sampling techniques as the cross-entropy method. The algorithm operates in a continuous loop, alternating between "local" and "global" phases until a set number of generations is reached:

1. **Initialize & Local Search:** Generate a pool of valid combinatorial configurations and apply a classical local search routine (or greedy algorithm) to optimize them based on a defined reward function.
2. **Filter:** Select the top-performing configurations with the highest scores to form an elite training dataset.
3. **Train (Global Phase):** Train the Makemore [] transformer architecture on this elite dataset so the model can learn the underlying structural patterns of the highly successful constructions.
4. **Sample:** Generate a new batch of candidate configurations by sampling tokens from the newly trained transformer.
5. **Refine:** Use these new candidates as starting seeds for the next round of local search.

By repeating this cycle, the algorithm gradually shifts its search towards regions that yield the highest rewards. The transformer continuously learns from the best local refinements, while the local search escapes local optima by starting from increasingly complex, globally-informed patterns.

1.5.2 State Representation

As per Section 1.4, the representation of our final problem set focuses on a permutation-based approach. We note, however, that the Cross-Entropy and PatternBoost approaches utilise binary string output types in their problem encodings. We

adopt this in the natural way. Subsequence problem instances simply utilise the binary representation of relevant integers. Consider an instance of an interval instersection problem, with the following configuration:

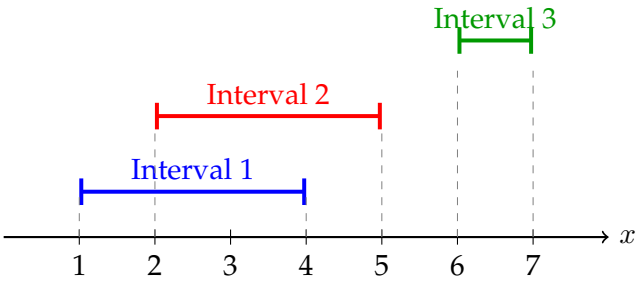


Figure 1.5: An example instance of an interval intersection problem.

We design a vector representation where position in the vector represents order of appearance, and value indicates the label of the relevant interval. Each interval label occurs twice, indicating it's start and end point. The representation then for the configuration depicted in Figure 1.5 is:

$$[1, 2, 1, 2, 3, 3]$$

These configurations are also translated from binary using the natural binary representations of each integer. We utilise this representation for rectangle-based problems too, by simply producing a second vector which defines the vertical configuration of the rectangles. This permutation formulation allows us to represent all valid intersection graphs, cutting out a huge number of equivalent solutions that would be included in a configuration based on defining rectangle co-ordinates instead.

To facilitate the local search mechanism in PatternBoost, we simply permute a small number of the integers in our state representations randomly. More complex local search approaches geared towards reward optimisation are difficult to design without introducing outsized computational complexity.

1.6 EAAI Problem Sets

In our final EAAI model AI assignment [], we include a number of problems for which our developed approach was successful.

1.6.1 Subsequence Problems

We chose subsequence problems as our introductory problems, due to their more easily understandable representation and reward system. These experiments have clear reward systems which do not suffer from sparse regions, making them more amenable to RL approaches.

The core of these tasks revolves around finding the longest increasing subsequence (LIS)—a sequence of elements in an array that are in strictly ascending order—or the longest decreasing subsequence (LDS), which seeks elements in strictly descending order. To create a more challenging, multi-objective problem, we also formulated a combined instance where the goal is to maximise both simultaneously.

For the individual tasks, the reward is simply the length of the instance's maximum length increasing or decreasing subsequence. For the combined task, the reward is calculated as the minimum of the two lengths, formulated as $\min(L_{inc}, L_{dec})$. This forces the agent to balance both properties rather than greedily optimising one at the expense of the other. The maximum possible reward for a list of length n is $\lceil n/2 \rceil$.

The representation of an instance for these problems is a simple permutation of integers from 1 to n. Note that this list is translated directly from the binary output sequence generated by the cross-entropy method, as described in Section 1.5.1.

For a sequence of size $n = 12$, the best possible value for the combined reward is 6, meaning the sequence contains both an increasing and a decreasing subsequence of length 4. An example of an optimal configuration that achieves this maximum reward is:

$$[12, 1, 11, 2, 10, 3, 9, 4, 8, 5, 7, 6]$$

Our implementation, utilizing the Cross-Entropy method, is capable of successfully solving these subsequence problems for lengths up to $n = 32$.[1]

[1]The code for this implementation is publicly available at:
`https://colab.research.google.com/drive/18_MqhFqt5INjuuxMtc_cbIvOh1xgpFUP?usp=sharing`

1.6.2 Rectangle-Based Problems

With a view towards MISR, we considered another problem varient based on rectangle intersections which had a less sparse reward landscape.

This problem involves arranging n red and n blue axis-aligned rectangles within a two-dimensional coordinate space. These rectangles can vary in thickness, size, and overall position. The goal is to find a configuration that satisfies two objectives:

- Maximising the intersections between differently-coloured rectangles.
- Minimising the intersections between same-coloured rectangles.

Balancing these objectives represents a challenging geometric problem for our RL agents.

To encode this problem, we utilize the state representation as outlined in Section 1.5.2. A rectangle's colour is assigned based on its label index: rectangles with odd labels are designated red, while those with even labels are designated blue.

The quality of a given configuration is evaluated using the following reward function:

$$\text{Objective} = \text{RBI} - \tau(\text{RRI} + \text{BBI}) \tag{1.1}$$

Where:

- **RBI** represents the number of red-blue rectangle intersections.
- **RRI** represents the number of red-red rectangle intersections.
- **BBI** represents the number of blue-blue rectangle intersections.
- τ is a dynamic penalty coefficient.

The penalty term τ is incorporated specifically to discourage intersections between rectangles of the same colour. During the initial phases of the algorithm, τ is set to a high value to heavily penalise these unwanted overlaps. As the training progresses, τ is gradually reduced to 1. This dynamic scheduling encourages the model to first learn configurations with minimal same-colour overlap before shifting its priority toward maximising the desirable red-blue intersections.

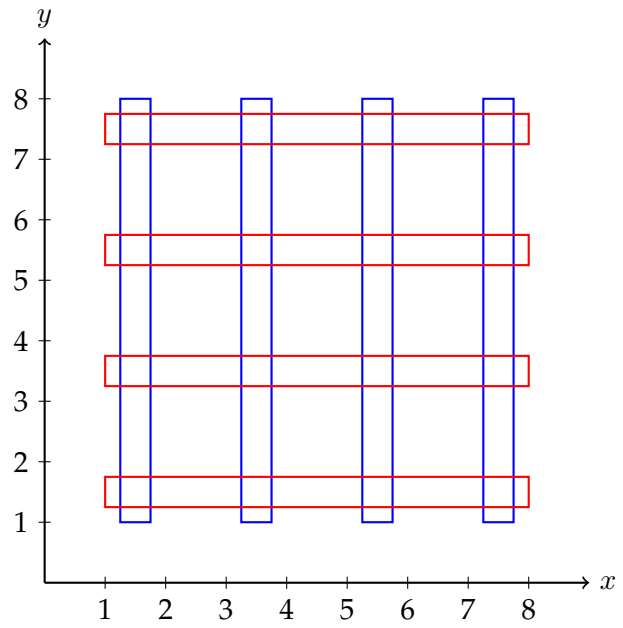


Figure 1.6: Optimal configuration for red and blue rectangle problem

We depict an optimal configuration for $n- = 4$ in Figure 1.6.Although this configuration may seem an obvious solution to a human, it was difficult in practice to develop an RL approach which can find this type of instance. For the MISR problem, we will see that instances with this global lattice configuration are relevant, so it was encouraging that our techniques could handle this problem as we built towards finding MISR instances with large integrality gaps.

1.7 Integrality Gap Instances for Maximum Independent Set of Rectangles

The culmination of our developed methodology focuses on the MISR problem. Even with effective RL agents and state representations, finding instances with large integrality gaps on this problem remains a difficult proposition. Reward signals are expensive to compute on this problem by definition, as the reward function requires solving an instance of MISR to optimality (as well as the LPR instance). Alternative reward functions which could somehow guide agents towards worst-case instances are unclear. Jitter-based alternatives introduce outsized computational overhead due to repeated ILP solver calls, and additional characteristics which may occur in a worst-

case instance are unclear. As such, we focus on the exact reward function: the ratio between LPR and ILP solution values.

At the outset of this project, the worst known instance integrality gap was the beautiful construction by Chuzoy et al, as described in Figure 1.1 (Section1.1).

When this work began, the largest known integrality gap was provided by the worst-case construction of Chuzoy et al., as seen earlier in Figure 1.1. The ratio of ILP to LPR solution size approaches $3/2$ as $n \to \infty$. However, a more recent work by Caoduro et al.[] has since increased the worst known gap, with a construction that approaches a ratio of 2 as $n \to \infty$.

Our first success with RL approaches on finding high integrality gap instances came with the PatternBoost algorithm. We perform these initial experiments with $n = 8$ up to $n = 32$ rectangles, and include in it's initial dataset some simple 5-cycles of rectangles, as well as some of the small Chuzoy instances. We discovered an instance with a ratio of exactly 1.5 with only x rectangles, as depicted in Figureřef

This instance surpasses the previously worst-known instances from Chuzoy which can only reach this ratio with an infinite number of rectangles. We examined this instance more carefully to investigate if it had any exploitable characteristics. This is more clear when depicted as a normal graph instance, which we show in Figure1.8.

In the original Wagner paper [], the author visually inspects some smaller constructions, and scales their characteristics to larger instances to generate counterexamples - our aim was similar here. We noticed that these plots could be described as a pair of cycles, one "inner" and one "outer" cycle, with edges between them such that the following properties hold.

- No triangles are created
- If a maximum independent set on one cycle is selected, it will preclude any nodes on the other cycle from being added.

If we denote inner and outer cycle sizes as a and b, we see how this yields an integrality ratio of $b/(a+b)$ (or $a/(a+b)$, the larger cycle size is the numerator), as the ILP solution has size $b/2$ and the LPR has size $(a + b)/2$. The triangle condition ensures that no clique constraints are violated by selecting all rectangles with a value of 0.5.

By seeding this instance into our PatternBoost initial dataset, further constructions

Figure 1.7: MISR configuration with 1.5 integrality ratio.

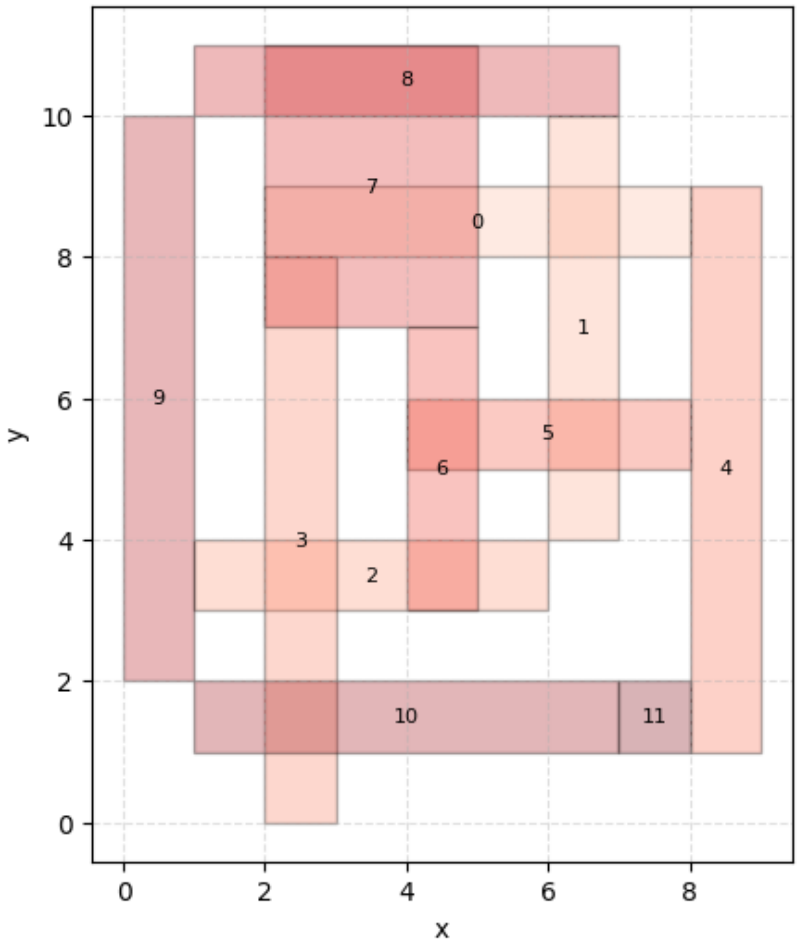
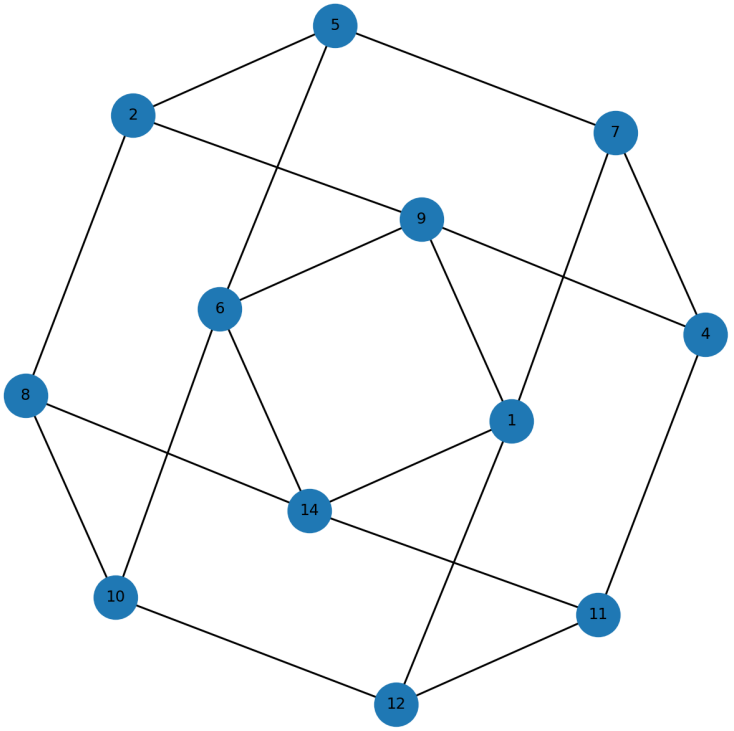


Figure 1.8: Illustration of 1.5 integrality ratio MISR configuration as a graph.

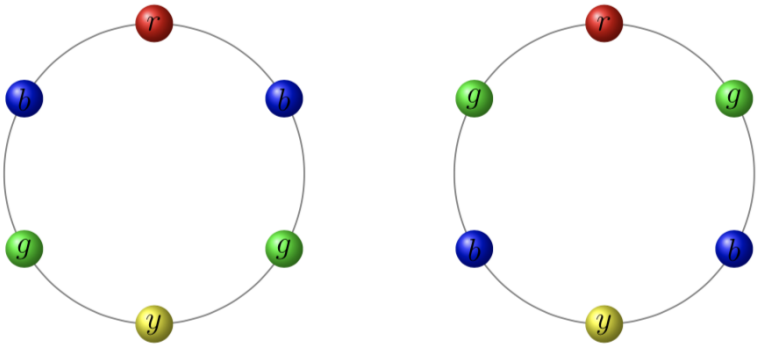


with these characteristics were created. We experimented with a formalised version of this construction by using ideas from necklace theory

necklace citations?

A k-ary necklace of length n is an equivalence class of n-character strings over an alphabet of size k, taking all rotations as equivalent. It represents a structure with n circularly connected beads which have k available colors (hence the moniker, "necklace"). In our setting, we refer to the inner cycle as our "necklace", and for each bead,

Figure 1.9: Example visualisation of two necklaces with $k = 3$ colours and length $n = 6$.



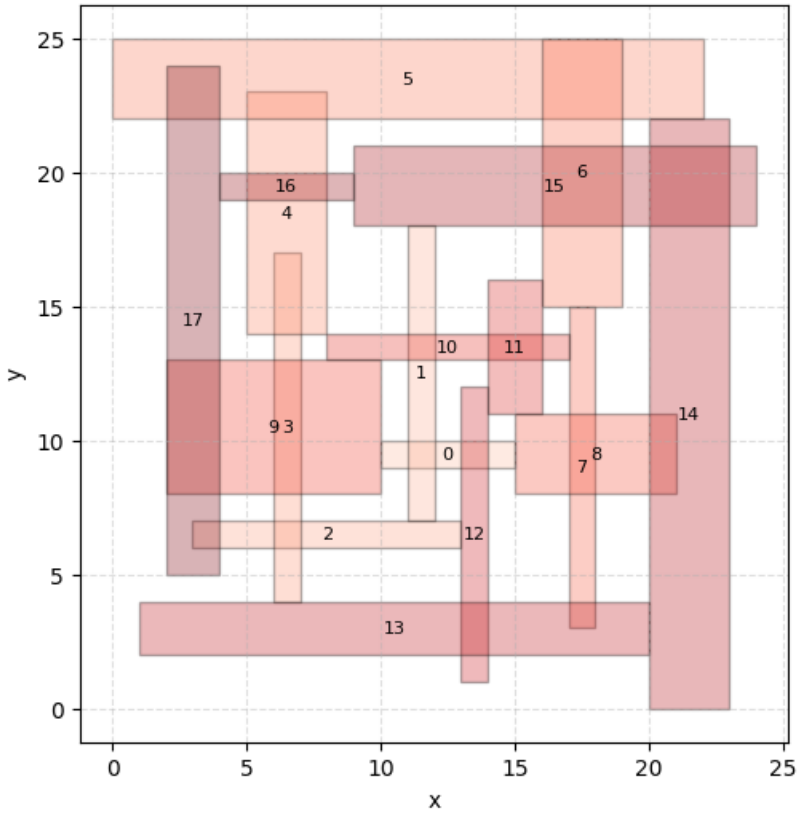
its "colour" represents the outer cycle nodes to which it is connected. The bead colour is where our formalisation requires some nuance, as these connections must retain the two properties listed above.

We leverage the parity of node labels to enforce the second constraint. If each inner node ("bead") is connected to one odd and one even node in the outer cycle, then a MIS on the inner cycle will be the maximal over the graph of both cycles. We can thus represent each bead "colour" as a tuple of integers, where each one describes the connection to one odd and one even outer cycle node. Rather than encoding this using node IDs directly, we capture a more broad set of configurations by using these tuples to denote a "shift" in node connections. A value of 0 means that a node i in the inner cycle will be connected to the node $(i \mod b)+a$ in the outer cycle. Each "bead colour" tuple will contain an even and odd numbered shift, ensuring that this MIS property is retained. Triangle checking is difficult to completely prevent on-the-fly, although some triangles are easy to prevent using modulo arithmetic. We can thus utilise the FKM algorithm [] to generated necklaces in constant amortised time, covering a huge space of similar graph structures with varying inner and outer cycle size.

These experiments yield a huge number of similar graphs with non-zero integrality gaps, including many instances with integrality ratios of 1.5. We had, however, hoped that configurations where $a = b$ could yield instances with the elusive integrality ratio of 2. This is not currently possible with instances of this type, due to an emergent strategy to select rectangles in the MISR when cycle sizes are equal. We illustrate an example where $a, b = 9$ in Figure 1.10.

By selecting a non-maximal independent set in the inner cycle, more nodes can be selected in the outer cycle, spoiling the projected integrality ratio for these instances.

Figure 1.10: MISR configuration with inner and outer cycles of length $a, b = 9$, with integrality ratio of 1.5



1.8 Discussion

We present an RL-centric framework which can generate geometric configurations of interest across a range of problems. A long period of exploratory work has underpinned our experimental design, culminating in a framework which combine two key components:

- Permutation-based problem representation
- Cross-entropy/PatternBoost RL agents

This approach works well on a range of simple geometric combinatorial optimisation problems including subsequence and rectangle-based problems, and has been used to discover a novel family of MISR problem instances with large integrality gaps, though not worse than the most recently discovered worst-case instances, which approach an integrality ratio of 2 as $n \to \infty$. However, this construction does not reach an integrality ratio of 1.5 until $n > 60$, whereas our approaches have this value with only $n = 12$ rectangles. Scaling the necklace-based approach beyond $a, b = 9$ is extremely computationally expensive, as the number of possible permutations exceeds xxx. Not all of these configurations are relevant, so further improvements to the necklace representation are required to facilitate larger scaling and faster instance enumeration.

We continue this work with further experiments in the RL framework, including seeding small instances from the recently discovered worst-case family of instances, as well as the new instances with integrality ratio of 1.5 found by our RL agents.

Our work has proven to be of independent interest in teaching applications. At $EAAI26$, our Model AI Assignment submission based on this approach was accepted upon review, and garnered interest from other educators in the area. Pre-existing teaching resources tended to focus on a range of RL agent applications to classic RL benchmark instances, whereas our approach inverts this. Our work exhibits the difficulty surrounding problem representation and a framework which succeeds in overcoming this, which is a key enabler for the adaptation of RL in novel problem settings.

BIBLIOGRAPHY

[1] Bossek, J. & Neumann, F. (2022). Exploring the feature space of tsp instances using quality diversity. In *Proceedings of the Genetic and Evolutionary Computation Conference*, 186–194.

[2] Charton, F., Ellenberg, J.S., Wagner, A.Z. & Williamson, G. (2024). Patternboost: Constructions in mathematics with a little help from ai. *arXiv preprint arXiv:2411.00566*.

[3] Do, A., Guo, M., Neumann, A. & Neumann, F. (2022). Analysis of evolutionary diversity optimization for permutation problems. *ACM Transactions on Evolutionary Learning*, **2**, 1–27.

[4] Do, A.V., Bossek, J., Neumann, A. & Neumann, F. (2020). Evolving diverse sets of tours for the travelling salesperson problem. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*, 681–689.

[5] Erdős, P., Rényi, A. & T Sós, V. (1966). On a problem of graph theory. *Studia Scientiarum Mathematicarum Hungarica*, **1**, 215–235.

[6] Gao, W., Nallaperuma, S. & Neumann, F. (2016). Feature-based diversity optimization for problem instance classification. In *International Conference on Parallel Problem Solving from Nature*, 869–879, Springer.

[7] Georgiev, B., Gómez-Serrano, J., Tao, T. & Wagner, A.Z. (2025). Mathematical exploration and discovery at scale. *arXiv preprint arXiv:2511.02864*.

[8] Ghebleh, M., Al-Yakoob, S., Kanso, A. & Stevanović, D. (2026). Reinforcement learning for graph theory, i: Reimplementation of wagner's approach. *Discrete Applied Mathematics*, **380**, 468–479.

[9] Mehrabian, A., Anand, A., Kim, H., Sonnerat, N., Balog, M., Comanici, G., Berariu, T., Lee, A., Ruoss, A., Bulanova, A. *et al.* (2023). Finding increasingly large extremal graphs with alphazero and tabu search. *arXiv preprint arXiv:2311.03583*.

[10] Nikfarjam, A., Bossek, J., Neumann, A. & Neumann, F. (2021). Computing diverse sets of high quality tsp tours by eax-based evolutionary diversity optimisation. In *Proceedings of the 16th ACM/SIGEVO Conference on Foundations of Genetic Algorithms*, 1–11.

[11] Nikfarjam, A., Bossek, J., Neumann, A. & Neumann, F. (2021). Entropy-based evolutionary diversity optimisation for the traveling salesperson problem. In *Proceedings of the Genetic and Evolutionary Computation Conference*, 600–608.

[12] Nikfarjam, A., Neumann, A. & Neumann, F. (2024). On the use of quality diversity algorithms for the travelling thief problem. *ACM Transactions on Evolutionary Learning and Optimization*, **4**, 1–22.

[13] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. *et al.* (2011). Scikit-learn: Machine learning in python. *J. of machine Learning research*, **12**.

[14] Pugh, J.K., Soros, L.B. & Stanley, K.O. (2016). Quality diversity: A new frontier for evolutionary computation. *Frontiers in Robotics and AI*, **3**, 40.

[15] Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A. *et al.* (2017). Mastering the game of go without human knowledge. *nature*, **550**, 354–359.

[16] Smith-Miles, K., Baatar, D., Wreford, B. & Lewis, R. (2014). Towards objective measures of algorithm performance across instance space. *Computers & Operations Research*, **45**, 12–24.

[17] Trinh, T.H., Wu, Y., Le, Q.V., He, H. & Luong, T. (2024). Solving olympiad geometry without human demonstrations. *Nature*, **625**, 476–482.

[18] Ventosa, L. & Knauer, C. (2023). *Learning-Powered CounterExample Search: Applying the Cross-Entropy Method to Spectral Graph Theory*. Bachelor's thesis, University of Potsdam, Potsdam, Germany.

[19] Wagner, A.Z. (2021). Constructions in combinatorics via neural networks. *arXiv preprint arXiv:2104.14516*.